

Engineering Document E02 — Database Engineering Blueprint

Product Name: HQ

Engineering Package: Version 1.0

Purpose

This document defines the complete database architecture for HQ.

The database is the foundation of the platform and is responsible for storing organizational data, user identities, executive operations, AI missions, assets, billing, analytics, and system events.

HQ adopts **PostgreSQL** as its relational database and **Prisma ORM** as the official database access layer.

Engineering Principles

The database must be:

- Scalable
- Secure
- Multi-tenant
- Auditable
- Highly normalized
- Type-safe
- Migration-driven
- Optimized for performance

The database should remain independent of business logic.

Technology Stack

Database

- PostgreSQL

ORM

- Prisma ORM

Migration Tool

- Prisma Migrate

Database Client

- Prisma Client

Seed Engine

- TypeScript Seed Scripts
-

Repository Structure

packages/

 database/

 prisma/

 schema.prisma

 migrations/

 seed.ts

 src/

 prisma.service.ts

 prisma.module.ts

 prisma.client.ts

 helpers/

extensions/

The database package is shared across the backend and future internal tooling.

Multi-Tenant Architecture

HQ follows an Organization-first model.

Organization

↓

Users

↓

Headquarters

↓

Executives

↓

Missions

↓

Assets

↓

Analytics

↓

Billing

Every business record belongs to an Organization.

No organization can access another organization's data.

Primary Database Entities

Authentication

- User
- Session

Organization

- Organization
- Headquarters
- OrganizationSettings

Executive System

- Executive
- ExecutiveMemory
- ExecutiveActivity

Mission System

- Mission
- MissionTask
- MissionComment
- MissionOutput
- MissionTimeline

Asset System

- Asset
- Folder
- AssetVersion

AI System

- Prompt
- PromptVersion
- AIRequest
- AIResponse

Billing

- Subscription
- Plan
- Invoice
- Payment

Analytics

- AnalyticsEvent
- DashboardMetric

Security

- AuditLog
- LoginHistory
- ApiKey

System

- Notification
- ActivityLog
- FeatureFlag

Common Fields

Every table should include:

- id
- createdAt
- updatedAt

Where applicable:

- deletedAt
- organizationId
- createdBy
- updatedBy

This provides consistency and auditability.

Primary Keys

All tables use:

UUID Version 7 (UUIDv7)

Advantages:

- Globally unique
- Ordered for improved index locality
- Suitable for distributed systems

IDs should never expose business information.

Foreign Keys

All relationships enforce foreign key constraints.

Examples:

Mission

→ Organization

Mission

→ User

MissionTask

→ Mission

ExecutiveActivity

→ Executive

Subscription

→ Organization

Referential integrity must be maintained.

Relationship Standards

One-to-One

Examples:

Organization

↔ Headquarters

One-to-Many

Examples:

Organization

↓

Users

Mission

↓

Tasks

Many-to-Many

Examples:

Users

↔ Organizations (future support)

Executives

↔ Missions

Join tables should be used where appropriate.

Prisma Modeling Standards

Every model should define:

- @@map()
- @map()

- `@@index()`
- `@@unique()`

Database naming remains stable even if Prisma model names evolve.

Naming Conventions

Prisma Models

PascalCase

Example:

User

Mission

Executive

Database Tables

snake_case

Examples:

users

mission_tasks

executive_memory

Columns

snake_case

Example:

created_at

organization_id

updated_by

Enums

PascalCase in Prisma

Mapped to PostgreSQL enum types.

Index Strategy

Indexes should exist for:

Primary Keys

Foreign Keys

Authentication Lookups

Organization Filtering

Mission Status

Executive Status

Created Date

Billing

Search Fields

Composite indexes should be added for high-frequency queries.

Soft Deletes

Business entities should support soft deletion.

Fields:

deletedAt

deletedBy

Records remain recoverable unless permanently removed through administrative processes.

Audit Fields

Every important business change should capture:

Created By

Updated By

Deleted By

Timestamp

This supports accountability and compliance.

Transactions

Prisma transactions should be used for operations involving multiple dependent writes.

Examples:

- Creating a mission with tasks
- Subscription upgrades
- Organization setup
- Executive memory updates

Partial writes must be avoided.

Database Constraints

Examples:

Email

Unique

Organization Slug

Unique

Subscription

One active subscription per organization

Mission Status

Restricted to defined enum values

Constraints enforce data integrity.

Prisma Enums

Core enums include:

UserRole

MissionStatus

ExecutiveRole

SubscriptionPlan

PaymentStatus

NotificationType

ExecutiveStatus

Enums should be centralized within the Prisma schema.

Migration Strategy

Every schema modification requires:

1. Update schema.prisma
2. Generate migration
3. Review SQL
4. Test locally
5. Commit migration
6. Deploy migration

Manual production schema changes are prohibited.

Seed Strategy

Seed data includes:

- Plans
- Default Executives
- Feature Flags
- Application Settings

Development seeds should never contain production data.

Performance Strategy

Optimize through:

- Efficient indexes
- Query optimization
- Pagination
- Cursor-based loading
- Connection pooling
- Query monitoring

Database performance should be measured continuously.

Prisma Client Usage

A single Prisma Client instance should be shared throughout the application.

Benefits:

- Efficient connection management
 - Reduced memory usage
 - Consistent configuration
-

Backup Strategy

Backups include:

Database

↓

Point-in-Time Recovery

↓

Encrypted Storage

↓

Restoration Testing

Recovery procedures should be verified regularly.

Security

The database should enforce:

- Least privilege
- Encrypted connections
- Secure credentials
- Audit logging

- Tenant isolation

Sensitive information must never be stored in plain text.

Future Expansion

The schema should accommodate:

- Team workspaces
- Marketplace
- Plugin ecosystem
- Enterprise organizations
- AI provider abstraction
- Workflow automation

Schema evolution should remain backward compatible whenever practical.

Success Criteria

The Database Engineering Blueprint succeeds when:

- Data integrity is maintained.
 - Prisma models remain type-safe.
 - Queries perform efficiently.
 - Migrations are predictable.
 - Organizations remain fully isolated.
 - The schema supports future product growth.
-

Conclusion

The HQ Database Engineering Blueprint establishes PostgreSQL and Prisma ORM as the authoritative data foundation for the platform.

By combining a normalized relational model, strong migration discipline, multi-tenant architecture, and Prisma's type-safe development workflow, HQ gains a scalable, secure, and

maintainable database capable of supporting both the hackathon MVP and long-term commercial expansion.